

Linear Probing

1. Concept

Definition

Linear Probing is an open addressing collision resolution technique where we sequentially search for the next empty slot.

$$h'(k) = k$$

$$h(k, i) = (h'(k) + i) \bmod m$$

Where:

- k = key
- i = probe number $(0, 1, 2, \dots)$
- m = table size

2. How Linear Probing Works

Step 1: Compute initial index

$$h(k, 0)$$

Step 2: If occupied, try

$$h(k, 1), h(k, 2), \dots$$

Move sequentially until an empty slot is found.

3. Simulation Example

Let:

$$m = 11$$

Insert keys:

10, 22, 31, 4, 15, 28, 17, 88, 89

Initial hash:

$$h(k, 0) = k \bmod 11$$

Step-by-Step Insertion (m = 11)

Hash function:

$$h'(k) = k$$

$$h(k, i) = (h'(k) + i) \bmod 11$$

Initial probe:

$$h(k, 0) = k \bmod 11$$

1) Insert 10

$$h(10, 0) = 10 \bmod 11 = 10$$

Index 10 is empty $\rightarrow$ Insert.

Probes = 1

2) Insert 22

$$h(22, 0) = 22 \bmod 11 = 0$$

Index 0 is empty $\rightarrow$ Insert.

Probes = 1

3) Insert 31

$$h(31, 0) = 31 \bmod 11 = 9$$

Index 9 is empty $\rightarrow$ Insert.

Probes = 1

4) Insert 4

$$h(4, 0) = 4 \bmod 11 = 4$$

Index 4 is empty $\rightarrow$ Insert.

Probes = 1

5) Insert 15

$$h(15, 0) = 15 \bmod 11 = 4$$

Index 4 is occupied $\rightarrow$ Collision.

Try next probe:

$$h(15, 1) = (15 + 1) \bmod 11 = 5$$

Index 5 is empty $\rightarrow$ Insert.

Probes = 2

6) Insert 28

$$h(28, 0) = 28 \bmod 11 = 6$$

Index 6 is empty $\rightarrow$ Insert.

Probes = 1

7) Insert 17

$$h(17, 0) = 17 \bmod 11 = 6$$

Index 6 occupied $\rightarrow$ Collision.

$$h(17, 1) = (17 + 1) \bmod 11 = 7$$

Index 7 empty $\rightarrow$ Insert.

Probes = 2

8) Insert 88

$$h(88, 0) = 88 \bmod 11 = 0$$

Index 0 occupied $\rightarrow$ Collision.

$$h(88, 1) = (88 + 1) \bmod 11 = 1$$

Index 1 empty $\rightarrow$ Insert.

Probes = 2

9) Insert 89

$$h(89, 0) = 89 \bmod 11 = 1$$

Index 1 occupied $\rightarrow$ Collision.

$$h(89, 1) = (89 + 1) \bmod 11 = 2$$

Index 2 empty $\rightarrow$ Insert.

Probes = 2

Insertion Summary Table

Key	Final Index	Probes
10	10	1
22	0	1
31	9	1
4	4	1
15	5	2
28	6	1
17	7	2
88	1	2
89	2	2

Total Probes = 13

$\alpha = \frac{9}{11} \approx 0.82$

4. Final Hash Table Visualization

0	1	2	3	4	5	6	7	8	9	10
22	88	89		4	15	28	17		31	10

5. Primary Clustering

Primary Clustering

Sequential probing (+1 each time) causes consecutive occupied cells to form clusters. Future insertions near that cluster require more probes.

Example cluster: indices 0–2.

6. Time Complexity

Let:

$$\alpha = \frac{n}{m}$$

Under uniform hashing assumption:

$$\text{Average successful search} \approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

As:

$$\alpha \rightarrow 1$$

Performance degrades rapidly.

7. Advantages and Disadvantages

Advantages

- Simple implementation
- No extra memory
- Cache-friendly

Disadvantages

- Primary clustering
- Performance sensitive to load factor
- Deletion requires special handling (lazy deletion)

Linear Probing = Simple but Clustering-Prone